

卒 業 論 文

Certified Actor Critic 法における 勾配の簡易計算法

2022TM049 佐藤 達志

指導教員 潮 俊光

2026 年 1 月

南山大学 理工学部 機械システム工学科

概要

システムの振る舞いが状態空間上で指定された安全領域の中に留まり続けるという制御仕様を安全仕様という。安全仕様を満たしつつシステムを安定化する制御法を学習する方法に certified actor critic(CAC) がある。この方法では、最初に安全仕様を満たすように強化学習を行い、その後、安全仕様と安定化に対して個別に critic ネットワークを使って評価し、その評価をもとに actor の更新を行う。このとき、各 critic ネットワークに対応する勾配から actor の更新に用いる勾配を最適化問題に帰着して決定している。本研究では、従来の CAC が採用する「安全学習 → 安定化学習」の二段階構造を廃し、安全報酬と性能報酬を学習初期から同時に扱う統合型の方策更新手法を提案する。提案手法では、安全度指標と学習進行度に基づいて安全クリティック勾配と性能クリティック勾配を動的に混合し、さらに性能勾配の過大更新を防ぐために対数縮小・ソフト縮小・相対上限・ハードクランプからなる多段階ノルム制御を導入する。これにより、安全性を維持しつつ性能改善を妨げない更新方向を生成できることを示す。Pendulum-v1 環境を用いたシミュレーションにより、提案手法が安全性の低下を抑えながら安定化性能を向上させることを確認する。

目次

第 1 章	はじめに	1
1.1	研究背景	1
1.2	研究目的と研究内容	1
第 2 章	強化学習	3
2.1	マルコフ決定過程	3
2.2	価値関数	3
2.3	強化学習アルゴリズム	4
2.4	深層強化学習	5
2.5	SAC	6
第 3 章	安全度と進行度に基づく勾配混合手法	8
3.1	CAC の二段階構造と Restricted Policy Update	8
3.2	提案手法の概要	9
3.3	安全度 $h(s)$ ・安全報酬 r_1 ・到達性能報酬 r_2	9
3.4	Safety Critic と Navigation Critic	11
3.5	安全勾配 W_1 と性能勾配 W_2 の算出	11
3.6	勾配方向の決定	12
3.7	勾配の大きさの調整	16
3.8	バリア関数パラメータ α のスケジューリングとエントロピー温度との関係	21
3.9	提案手法の学習アルゴリズム	24
第 4 章	提案手法の検証	28
4.1	実験設定	28
4.2	学習結果	29
4.3	考察	29
第 5 章	おわりに	30
5.1	まとめ	30
5.2	今後の課題	30
	謝辞	31

第 1 章

はじめに

1.1 研究背景

強化学習は、試行錯誤を通じて最適な行動方策を自律的に獲得する手法であり [1], ロボティクスや制御など様々な分野で応用されている [2]. 特に, 深層学習を用いた深層強化学習は, 高次元・連続状態を扱える手法として発展してきた [3, 4, 5].

深層強化学習の代表的手法である Deep Q-Network (DQN) は, 状態を入力として各行動の Q 値を出力する深層ニューラルネットワークを用い, 経験再生によって学習性能を向上させた [6]. 一方, 連続値の行動空間を扱う問題に対しては, 方策勾配定理 [7] に基づくアクタークリティック法が有効であり, その拡張として TRPO [8] や PPO [9], さらに探索と利用のバランスを考慮した SAC [10] が提案されている.

しかし, 従来の強化学習アルゴリズムは, 探索過程において安全性を考慮せず行動を試みるため, 実行時に危険な行動を選択する可能性がある. これは, 制御対象や周辺環境に損害を与える恐れがあり, セイフティクリティカルなシステムへの適用を困難にしている. この問題に対し, 制約付きマルコフ決定過程 (CMDP) に基づくセーフ強化学習など, 安全性を考慮した手法が提案されている [11, 12].

一方, 制御工学分野では, 安全性を保証する手法としてバリア関数を用いた安全制御が注目されており [13, 14], 離散時間系への拡張も進められている [15, 16]. 近年では, これらを強化学習と組み合わせた手法として, バリア関数を用いたセーフ強化学習が提案されている [17].

その一例として, 安全性と安定化を異なるネットワークで評価し, それらの勾配に基づいて方策更新を行う certified actor-critic (CAC) が提案されている [18]. CAC では, アクター更新時に最適化問題を解くことで安全性と性能の両立を図っているが, この最適化計算は学習の計算負荷を増大させる要因となる.

1.2 研究目的と研究内容

本研究では, CAC における勾配計算を簡易化する方法を提案し, 安全性を維持しつつ学習性能および計算効率の向上が可能であるかを検討する.

本論文の構成は以下の通りである. 第 2 章では SAC 概要を説明する. 第 3 章では CAC の勾配の

簡易計算法を 3 つ提案し，シミュレーションにより提案手法の有効性を検証する．第 4 章では，提案手法で確認された問題点を改良した手法を提案し，シミュレーションにより検証する．

第2章

強化学習

2.1 マルコフ決定過程

マルコフ決定過程は、 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \tau_0, \tau, \rho, T)$ で定義される。ただし、各記号の意味は以下のとおりである。

- ・ $\mathcal{S}$ は環境の取りうる状態の集合である。有限集合の場合もあれば、連続的な集合の場合もある。
- ・ $\mathcal{A}$ は環境が受け付ける行動の集合である。有限集合の場合もあれば、連続的な集合の場合もある。
- ・ $\tau_0 \in \Delta(\mathcal{S})$ は時刻0における状態である初期状態 S_0 の従う確率分布であり、初期状態分布という。すなわち

$$\tau(s_0) := \mathbb{P}[S_0 = s_0]$$

が成り立つ。

- ・ $\tau_a(\cdot|s) \in \Delta(\mathcal{S}) (s \in \mathcal{S}, a \in \mathcal{A})$ は状態 s で条件つけたもとで、行動 a に応じて、次の時刻の状態 $s' \in \mathcal{S}$ の確率 $\tau_a(s'|s)$ 定めるもので、状態遷移分布という。
- ・ $\rho_a(\cdot|s) \in \Delta(\mathbb{R}) (s \in \mathcal{S}, a \in \mathcal{A})$ は状態 s で条件付けたもとで、行動 a に応じて、環境から得られる報酬の従う確率分布を定めるもので、報酬分布という。
- ・ $T \in \mathbb{N} \cup \{\infty\}$ はマルコフ決定過程の状態遷移の回数を規定するもので、時間長という。

2.2 価値関数

状態価値関数 $V^\pi(s)$ を次のように定義する。 $\gamma \in [0, 1]$ を割引率とする。

$$v^\pi(s) := \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t | S_0 = s \right]$$

状態価値関数は、方策 π に従って状態 s から行動したときに得られる期待累積報酬を表す。すなわち、ある状態から始めたときに「どれほど良い未来が期待できるか」を定量化する指標である。また、方策 π に対する状態価値関数は次の関係式を満たす。

$$V^\pi(s) = \mathbb{E}_{\substack{S' \sim \tau_a(\cdot|s) \\ A \sim \pi(\cdot|s)}} [r(s, A) + \gamma V^\pi(S')]$$

これをベルマン方程式と呼ぶ.

最適状態価値関数 $V^*(s)$ はすべての方策の中で最も高い期待累積報酬を与える価値関数であり, 次の関係式を満たす.

$$V^*(s) = \max_{a \in \mathcal{A}} \mathbb{E}_{S' \sim \tau_a(\cdot|s)} [r(s, a) + \gamma V^*(S')]$$

これをベルマン最適方程式と呼ぶ.

次に状態価値関数の拡張として, 状態 s で行動 a を選択した場合に得られる期待累積報酬を表す行動価値関数 $Q^\pi(s, a)$ を次のように定義する.

$$Q^\pi(s, a) := \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right]$$

最適行動価値関数 $Q^*(s, a)$ は状態 s で行動 a を選択したときに得られる期待累積報酬の最大値として次式のように定義される.

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a) \quad (2.1)$$

$$= \mathbb{E}_{S' \sim \tau_a(\cdot|s)} [r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(S', a')] \quad (2.2)$$

最適状態価値関数との関係は次式で与えられる.

$$\begin{aligned} V^*(s) &= \max_{a \in \mathcal{A}} Q^*(s, a) \\ Q^*(s, a) &= \mathbb{E}_{S' \sim \tau_a(\cdot|s)} [r(s, A) + \gamma V^*(S')] \end{aligned}$$

したがって, 最適方策 π^* は次式で定義される.

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q^*(s, a)$$

すなわち, ベルマン最適方程式を解くことで最適価値関数が得られ, その結果として最適方策を導出できる.

2.3 強化学習アルゴリズム

強化学習は試行錯誤を通じて最適な行動方策を自律的に獲得できる手法である. 環境は初期状態分布, 状態遷移分布, 報酬分布といった確率的な構造を持つが, これらは一般に未知である. そのため, エージェントは環境の確率分布を直接利用することなく, 試行錯誤を通じて得られる経験から方策を改善していく必要がある. すなわち, 強化学習は「未知の環境において, 経験に基づき最適な行動を学習する」ための方法論である. この強化学習の代表的な手法の一つとして Q 学習がある. Q 学習は, 環境の確率分布を仮定することなく, 行動価値関数を逐次更新することで最適方策を獲得するアルゴリズムである. 以下では, Q 学習を例に挙げて強化学習アルゴリズムの基本的な仕組みを説明する.

Q 学習とは, 時間差分学習の枠組みで最適行動価値関数を求めるアルゴリズムである. 最適行動価値関数 (2.2) の推定量を時間差分学習の考えを適用すると, 状態, 行動, 報酬, 次状態の経験 (s, a, r, s') に対して, 次の更新式を示す. $\alpha > 0$ は学習率と呼ばれる量で, アルゴリズムの更新回数に応じて小さくなるような値を用いる.

$$\hat{q}^*(s, a) \leftarrow (1 - \alpha) \hat{q}^*(s, a) + \alpha (r + \gamma \max_{a' \in \mathcal{A}} \hat{q}^*(s', a')) \quad (2.3)$$

これをもとに最適行動価値関数を推定していく．つまり， Q 学習は『現在の推定値』と『一步先の報酬＋次状態の最大価値』を繰り返し近づけることで最適解に収束する．

2.4 深層強化学習

Q 学習は状態行動空間が巨大であったり連続空間の場合は表形式で Q 値を推定することが困難となる．この問題を克服するために Q 値や価値関数を近似する関数近似が導入され，さらにニューラルネットワークの表現力を活用することで深層強化学習が発展した．深層強化学習は強化学習の枠組みにディープラーニングを導入することで，従来の手法では困難であった高次元・連続的な状態空間を扱えるようにした手法でコンピュータに非常に高度な認識，判断能力を身につけることができる．

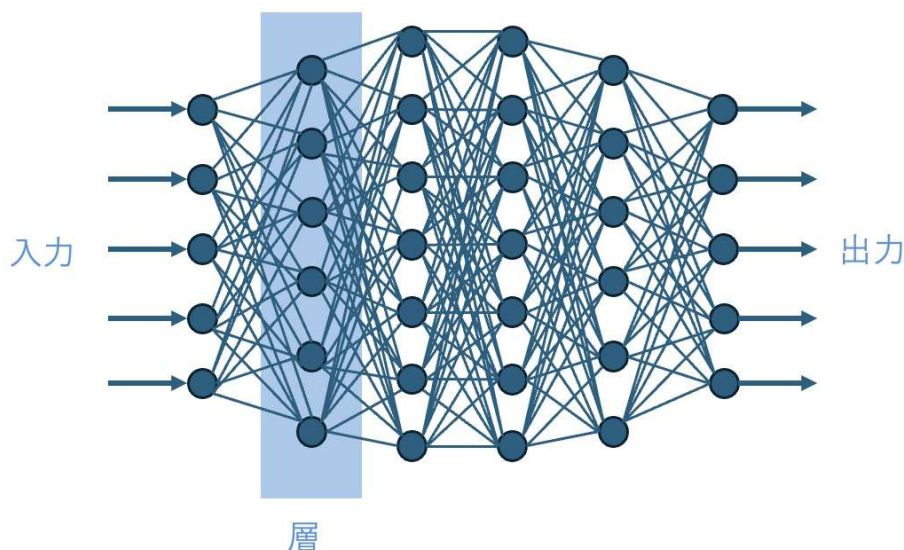


図 2.1 多層からなるニューラルネットワーク

図 2.1 は，ディープラーニングで使われる多層のニューラルネットワークの例である．複数のネットワークに入力があり，ネットワークを通して出力となるがこのネットワーク，いわゆるニューラルネットワークはまるで人の脳のように柔軟に学習することができる．ディープニューラルネットワークは，複数の特徴量を入力層で受け取り，隠れ層を通じて非線形変換を繰り返すことで出力を生成する構造を持つ．出力は目的の値と比較され，その誤差が計算される．学習過程では，この誤差を逆伝播させることで各層の重みやバイアスを更新し，誤差が徐々に小さくなるように最適化が行われる．すなわち，ディープニューラルネットワークは「入力された複数の特徴量を基に出力を予測し，誤差逆伝播によって内部パラメータを調整する」ことで複雑な関数近似を実現する．

2.5 SAC

2.5.1 準備

連続 MDP の定義：連続状態空間 $\mathcal{S}$, 連続行動空間 $\mathcal{A}$ を持つマルコフ決定過程 $(\mathcal{S}, \mathcal{A}, p, r)$ を考える。ただし, $p: \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, \infty)$ は遷移確率密度関数, $r: \mathcal{S} \times \mathcal{A} \rightarrow [r_{min}, r_{max}]$ は有界な報酬を表す。

2.5.2 最大エントロピー強化学習

標準的な強化学習の目的関数は $\sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t)]$ である。最大エントロピー強化学習では方策エントロピー項を追加した以下の目的関数を最大化する方策を求める。

$$\sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))] \quad (2.4)$$

ここで, 方策エントロピーは

$$\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a_t \sim \pi} \log \pi(a_t|s_t) \quad (2.5)$$

であり, $\alpha > 0$ は温度パラメータである。

2.5.3 ソフト方策反復

ソフト Q 値を次のように定義する。 $\gamma \in [0, 1]$ を割引率とする。

$$Q^\pi(s, a) = r_0 + \mathbb{E}_{\tau \sim \rho_\pi, s_0=s, a_0=a} \left[\sum_{t=1}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right] \quad (2.6)$$

方策 π と Q 関数 Q に対して修正ベルマンバックアップ作用素 $\mathcal{T}^\pi$ とソフト価値関数 V^π を以下のよう定義する。

$$\begin{aligned} \mathcal{T}^\pi Q(s_t, a_t) &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V^\pi(s_{t+1})] \\ V^\pi &= \mathbb{E}_{a_t \sim \pi} Q(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \end{aligned} \quad (2.7)$$

方策評価：方策 π を与える。初期ソフト Q 関数 Q_0^π に対して反復 $Q_{t+1}^\pi = \mathcal{T}^\pi Q_t^\pi$ によって Q_t^π は Q^π に収束する。

方策改善：方策改善ステップでは, 更新されたソフト Q 値 Q^π の指数に比例するように各状態において次式に従って方策を改善する。

$$\pi_{new} = \operatorname{argmin}_{\pi' \in \Pi} D_{KL}(\pi'(\cdot|s_t) || \frac{\exp(\frac{1}{\alpha} Q^{\pi_{old}}(s_t, \cdot))}{Z^{\pi_{old}}(s_t)}) \quad (2.8)$$

D_{KL} はカルバック・ライブラー情報量である。この更新によって全ての (s, a) で $Q^{\pi_{new}}(s, a) \geq Q^{\pi_{old}}(s, a)$ が成り立つ。ソフト方策評価と改善を繰り返すことで, 最適な最大エントロピー方策に収束する。

SAC での実用的拡張：大規模な連続領域を対象とするときはソフト方策反復に対して実用的な近似を考える必要があり, ソフト Q 関数と方策の両方に対して関数近似器を用いる. Q 関数 $Q_\theta(s_t, a_t)$ と方策 $\pi_\theta(a_t|s_t)$ を考える. ソフト Q 関数のパラメータは以下のソフトベルマン残差を最小化するように更新する.

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim P} [V_\theta(s_{t+1})]))^2 \right] \quad (2.9)$$

方策パラメータの学習は期待 KL 情報量の最小化によって行う. 勾配が計算できるようにするために再パラメータ化トリックを使い行動 a_t を $a_t = f_\phi(\epsilon_t; s_t)$ として更新式を以下のようにする.

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}(0,1)} [\alpha \log(\pi_\phi(f_\phi(\epsilon_t; s_t)|s_t)) - Q_\theta(s_t, f_\phi(\epsilon_t; s_t))] \quad (2.10)$$

SAC ではこれらを利用し, リプレイバッファからのオフポリシー学習を用いてサンプル効率を大幅に向上させる. Critic には二つの Q ネットワークを使い, ソフトベルマン残差を最小化することで過大評価バイアスを抑制する. Actor はガウス方策と再パラメータ化トリックに基づく KL 最小化損失で更新し, 方策の決定性とランダム性の最適なバランスを実現する. さらに, 目標エントロピーを定めて温度パラメータを双対勾配降下で自動調節し, 動的に探索率を制御する.

第 3 章

安全度と進行度に基づく勾配混合手法

3.1 CAC の二段階構造と Restricted Policy Update

本研究の出発点は、従来の Certificated Actor-Critic (CAC) [18] が採用する学習構造とその課題を明らかにすることにある。

CAC は、離散時間制御バリア関数 (Discrete-time Control Barrier Function) を用いて安全集合 $\mathcal{C} = \{s \in \mathbb{R}^n : h(s) \geq 0\}$ を定義し、

$$h(s_{t+1}) + (\alpha_0 - 1)h(s_t) \geq 0, \quad 0 < \alpha_0 < 1 \quad (3.1)$$

を満たす限り、系が安全集合内に留まり続けること (forward invariance) を保証する。CAC はこの性質を用いて、学習を次の 2 段階に分離する。

Stage 1 (Safety Critic Construction) : 安全報酬

$$r_1(s_t, a_t) = \exp(\min(h(s_{t+1}) + (\alpha_0 - 1)h(s_t), 0)) \quad (3.2)$$

のみを用いて方策と安全クリティックを学習する。CAC はこの安全クリティック Q_1^π 自体が「安全性の定量的な証明書 (safetycertificate)」として機能することを理論的に示している (Theorem 1)。

Stage 2 (Restricted Policy Update) : 到達性能報酬 r_2 を導入し、安全性を損なわないように方策を更新する。具体的には、Actor の更新方向 ∇_θ を、安全クリティックの勾配 $W_1 = \nabla_\theta J_1(\theta)$ と到達性能クリティックの勾配 $W_2 = \nabla_\theta J_2(\theta)$ を用いた次の制約付き最適化問題の解として決定する。

$$\nabla_\theta = \arg \max_e e \cdot W_2 \quad \text{s.t.} \quad e \cdot W_1 \geq 0, \quad |e| \leq |W_2| \quad (3.3)$$

これは「安全クリティックの勾配方向を後退させない ($e \cdot W_1 \geq 0$) 範囲で、到達性能勾配 W_2 との内積を最大化する方向 e を選ぶ」という凸最適化問題であり、CAC はこれを毎回の Actor 更新ステップで解く。

この構造が抱える課題は主に次の 2 点である。

計算コスト : (3.3) は凸最適化問題であり、Actor の更新のたびにこれを解く必要がある。CAC の原論文自身も「この最適化計算が学習の計算負荷を増大させる要因となる」と述べている [18,

Introduction / 本論文 1.1 節参照]. 本研究では, Stage 1 から Stage 2 への切替に伴って, 学習に用いる報酬構造および Critic 構成が変化することを, 学習過程上の課題の一つとして捉える. 本研究は, この 2 点に着目し, (1) 段階の不連続性を解消するために二段階構造を撤廃し安全性・性能を学習初期から同時に扱う構成へ変更すること, (2) 計算コストを解消するために (3.3) の最適化計算を, 勾配間の内積およびノルムに基づいて更新方向を直接決定するヒューリスティックな勾配計算則へ置き換えること, の 2 つを提案する. この 2 点は独立した簡略化であり, 後述するように Cleaned.ipynb 上でもそれぞれ別の実装箇所 (Stage 分岐条件の無効化, および *restricted_direction* 関数) に対応している.

3.2 提案手法の概要

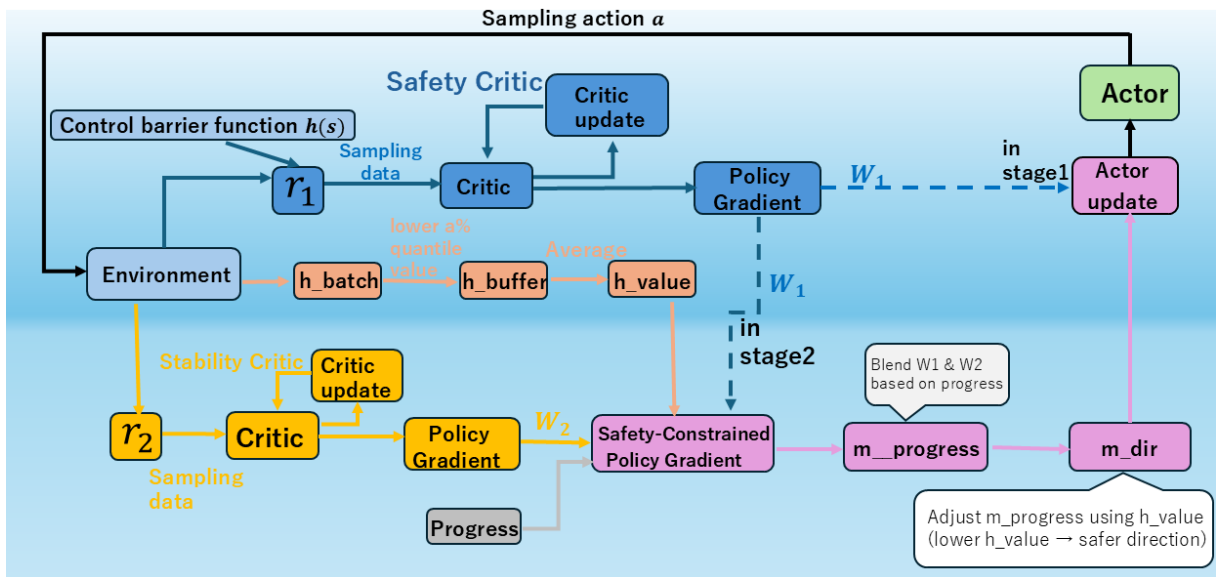


図 3.1 Gradient Blending Process

CAC の枠組みと対応付けると, 本提案は次のように整理できる.

対象タスクは Pendulum-v1 (状態 $s = [\cos \theta, \sin \theta, \dot{\theta}]$, 連続行動 $a \in [-2, 2]$) であり, Actor・Critic はいずれも中間層 256 ユニットの多層ニューラルネットワークで構成される (ネットワーク構成の詳細は第 4 章 実験設定で述べる).

以降の節では, この概要を構成する各要素を「なぜ必要か → 数式 → 変数の意味 → アルゴリズム中での使用箇所」の順に説明し, 3.9 節で学習アルゴリズム全体を擬似コードとしてまとめる.

3.3 安全度 $h(s)$ ・安全報酬 r_1 ・到達性能報酬 r_2

3.3.1 安全度 $h(s)$ Pendulum-v1 の状態 $s = [\cos \theta, \sin \theta, \dot{\theta}]$ のうち, 角速度 $\dot{\theta}$ に着目し, 安全度を次式で定義する (SoftActorCriticModel.h()).

表 3.1 CAC と本提案手法の対応関係

CAC の構成要素	本提案での扱い
Stage 1 (安全のみで方策更新)	分岐自体はコード上に残すが、発火条件を無効化し、実質的に Episode 1 から Stage 2 相当の処理を行う
Stage 2 の制約付き最適化 (3.3)	ヒューリスティックな勾配計算則 <code>restricted_direction</code> (3.6・3.7 節) に置き換える
安全報酬 r_1	CAC の式 (3.2) をそのまま踏襲する
到達性能報酬 r_2	独自の評価関数ではなく、環境 (Gymnasium Pendulum-v1) が返す標準報酬を用いる
<i>SafetyCritic/NavigationCritic</i>	いずれも Twin-Q 構成 (ClippedCriticNet) を用いる SAC 準拠の Critic として実装する

$$h_{\text{raw}}(s) = \dot{\theta} + \delta, \quad h_{\text{pos}}(s) = \min h_{\text{raw}}(s), h_{\text{max}}, \quad h(s) = \frac{h_{\text{pos}}(s)}{h_{\text{max}}} \quad (3.4)$$

ここで $\delta = h_{\text{max}} = 2.0$ である。安全側 ($\dot{\theta} > -\delta$) では $h(s)$ は上限 1 に飽和し、危険側 ($\dot{\theta} \leq -\delta$) では下限を持たず線形に負の方向へ発散する。したがって、 $h(s)$ は角速度に基づく非対称な安全度指標として機能する。

3.3.2 安全報酬 r_1 安全報酬は、CAC において用いられている指数型の安全報酬の設計を踏襲し、次式で定義する。

$$r_1(s_t, a_t) = \exp(\min(h(s_{t+1}) + (\alpha - 1)h(s_t), 0)) - 1 \quad (3.5)$$

なお、このコードでは加える定数として -1 が付いており、 $r_1 \in (-1, 0]$ の範囲を取る。また式中の α は安全条件における減衰率を表すパラメータであり、3.8 節で述べるスケジューリングにより毎エピソード変化する値が用いられる。

研究上の意味：この式自体は CAC 論文の式 (12) (指数正規化による安全報酬) と数式上完全に一致しており、本提案が独自に導入したものではなく、CAC の設計を踏襲した部分である。本研究の新規性は、この報酬をどう使うか (後述の勾配混合・スケジューリング) にある点を明確にしておく。

3.3.3 到達性能報酬 r_2 到達性能報酬には、独自に設計した評価関数ではなく、実験環境である Gymnasium の Pendulum-v1 が返す標準報酬を用いる。Pendulum-v1 では、角度、角速度および入力トルクに基づいて報酬が計算され、次式で表される。

$$r_2(s_t, a_t) = -(\theta_{\text{norm}}^2 + 0.1\dot{\theta}^2 + 0.001u^2) \quad (3.1)$$

(θ_{norm} は $(-\pi, \pi]$ に正規化した角度、 u は印加トルク) で与えられ、本提案手法固有の設計ではなく、環境側の既定義報酬をそのまま採用している。

以上より、本研究では、安全性を式 (3.5) の安全報酬によって評価し、到達性能を式 (3.6) の環境報

酬によって評価する．以降では，これらの報酬から得られる Safety Critic および Navigation Critic の勾配を用いて，Actor の更新方向および更新量を決定する．

3.4 Safety Critic と Navigation Critic

安全報酬 r_1 と到達性能報酬 r_2 に対応する行動価値関数を，それぞれ Safety Critic(`critic_safe`), Navigation Critic (`critic_net`) として個別に学習する．いずれも SAC に準拠した Twin-Q (`ClippedCriticNet`) 構成であり，過大評価バイアスを抑えるために 2 つのネットワークの最小値を用いる．

ソフトベルマンターゲットは次式で与えられる．

$$y_{\text{nav}} = r_2 + \gamma \mathbb{I}_{\{\text{not done}\}} (\min(Q_1^t, Q_2^t) - \alpha \log \pi(a'|s')) \quad (3.7)$$

$$y_{\text{safe}} = r_1 + \gamma \mathbb{I}_{\{\text{not done}\}} \min(Q_{1,\text{safe}}^t, Q_{2,\text{safe}}^t) \quad (3.8)$$

ここで， Q_1^t および Q_2^t は Navigation Critic のターゲットネットワーク， $Q_{1,\text{safe}}^t$ および $Q_{2,\text{safe}}^t$ は Safety Critic のターゲットネットワークを表す．また， γ は割引率， $\pi(a'|s')$ は次状態 s' における方策の行動確率密度， α はエントロピー正則化の温度パラメータである．

損失関数は，それぞれの Twin-Q ネットワークについて目標値との平均二乗誤差（MSE）を計算し，その和を用いる．各 Critic は，対応する学習率 η_Q に基づく勾配降下によって更新される．また，ターゲットネットワークは，更新率 $\tau = 0.005$ を用いたソフト更新により更新する．

なお，Navigation Critic の目標値 y_{nav} に含まれる $-\alpha \log \pi(a'|s')$ は，SAC の最大エントロピー強化学習に由来するエントロピー項である．一方，Safety Critic の目標値 y_{safe} にはこのエントロピー項は含まれない．本研究における α の設定については 3.8 節で述べる．

3.5 安全勾配 W_1 と性能勾配 W_2 の算出

Actor 側では，Navigation Critic に基づく性能側の Actor 損失（以下，stability loss）と，Safety Critic に基づく安全側の Actor 損失（以下，safety loss）をそれぞれ独立に逆伝播し，Actor パラメータに対する 2 つの勾配ベクトルを算出する．性能側の Actor 損失は次式で定義する．

$$J_{\text{stab}}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [\alpha \log \pi_\theta(a|s) - \min(Q_1(s, a_\theta), Q_2(s, a_\theta))] \quad (3.9)$$

一方，安全側の Actor 損失は次式で定義する．

$$J_{\text{safe}}(\theta) = -\lambda_{\text{safe}} \mathbb{E}_{s \sim \mathcal{D}} [\min(Q_{1,\text{safe}}(s, a_\theta), Q_{2,\text{safe}}(s, a_\theta))] \quad (3.10)$$

ここで， $\mathcal{D}$ は Replay Buffer から得られる状態の分布， θ は Actor のパラメータ， λ_{safe} は安全側の勾配の寄与を調整する係数である．

コード上では，stability loss に対して `stability_loss.backward(retain_graph=True)` を実行することで得られた Actor パラメータの勾配を平坦化し，性能勾配 W_2 として取得する．同様に，

safety loss に対して `safety_loss.backward(retain_graph=True)` を実行し、得られた勾配を安全勾配 W_1 として取得する。これらはそれぞれコード中の `grad_st` および `grad_sa` に対応する。`retain_graph=True` を指定することで、計算グラフを保持したまま 2 回の逆伝播を行い、同一の Actor パラメータに対する性能側と安全側の勾配を個別に取得している。

$$W_1 = \nabla_{\theta} J_{\text{safe}}(\theta), \quad W_2 = \nabla_{\theta} J_{\text{stab}}(\theta) \quad (3.11)$$

本提案手法では、実験条件として `args['stage1_episodes'] = 0` を設定している。また、`episode_index` は 1 から始まるため、

$$\text{episode_index} \leq \text{args['stage1_episodes']} \quad (3.2)$$

という Stage 1 の条件は、いずれのエピソードにおいても成立しない。したがって、Stage 1 の Actor 更新処理は実行されず、すべてのエピソードで Stage 2 側の処理、すなわち W_1 と W_2 を用いた更新方向の決定が行われる。

このように、本提案手法では CAC の二段階構造を維持するのではなく、学習開始時から安全性と到達性能の両方を考慮して Actor を更新する構成としている。なお、Stage 1 に関する分岐および `stage1_episodes` は CAC の実装から引き継いだものであり、本提案手法における本質的な処理ではない。

得られた安全勾配 W_1 と性能勾配 W_2 は、次節で述べる `restricted_direction` 関数に入力される。同関数では、これらの勾配の内積やノルム、さらに安全度および進行度を用いて更新方向とその大きさを決定し、最終的な Actor 更新に用いる。

3.6 勾配方向の決定

本提案手法では、Safety Critic および Navigation Critic から得られる安全勾配 W_1 と性能勾配 W_2 を直接加算するのではなく、両者の幾何学的関係と学習の進行度、さらに現在の安全度を考慮して、Actor の更新方向を決定する。本節では、`restricted_direction` 関数における処理の順序に従い、性能勾配 W_2 の縮小、混合係数 m_1, m_2 の算出、学習進行度による補間、バッチ安全度に基づく安全ゲート、および最終的な更新方向の算出について説明する。なお、ここで求められる更新方向は、CAC で用いられる制約付き最適化問題を直接解いたものではなく、その制約条件における勾配の幾何学的関係を参考として構成したヒューリスティックな勾配計算則である。

3.6.1 W_2 の対数縮小

性能勾配 W_2 は、学習初期や状態が不安定な場合に大きなノルムを持つことがある。このような性能勾配をそのまま混合計算に用いると、安全勾配 W_1 との内積や後段の混合係数の計算において、勾配のスケールが過度に影響する可能性がある。そこで、本手法では混合係数を計算する前に、 W_2 のノルムに対して対数関数を用いた縮小処理を行う。

まず、 W_2 のノルムに基づいて縮小係数 β を

$$\beta = \frac{1}{1 + \max\{\log_{10}(\|W_2\| + 1), 0\}} \quad (3.3)$$

と定義する．このとき，縮小後の性能勾配 $W_{2,\text{use}}$ を

$$W_{2,\text{use}} = \beta W_2 \quad (3.4)$$

とする．

式 (3.3) より， $\|W_2\|$ が大きくなるほど β は小さくなり，性能勾配の大きさが緩やかに抑制される．この処理では性能勾配の方向そのものを変更するのではなく，その大きさのみを縮小する．

実装では，`use_log10_shrink=True` および `log1p_variant=True` の場合にこの処理を適用している．具体的には，

$$\log_norm2 = \log_{10}(\|W_2\| + 1) \quad (3.5)$$

および

$$\text{shrink} = \frac{1}{1 + \max(\log_norm2, 0)} \quad (3.6)$$

を計算し，

$$W_{2,\text{use}} = \text{shrink} W_2 \quad (3.7)$$

としている．

3.6.2 混合係数 m_1 および m_2 の算出

次に，安全勾配 W_1 と縮小後の性能勾配 $W_{2,\text{use}}$ の幾何学的関係に基づいて，2つの混合係数 m_1 および m_2 を算出する．本手法では，両勾配が張る空間において，一方の勾配との内積が0となる混合方向を基準として用いることで，安全性と性能のトレードオフを考慮した更新方向を構成する．

$$e = mW_1 + (1 - m)W_{2,\text{use}} \quad (3.8)$$

を考える．

まず， e が安全勾配 W_1 と直交する条件

$$e \cdot W_1 = 0 \quad (3.9)$$

を考える．式 (3.8) を式 (3.9) に代入すると，

$$m\|W_1\|^2 + (1 - m)(W_1 \cdot W_{2,\text{use}}) = 0 \quad (3.10)$$

となる．これを m について解くことで，

$$m_1 = \frac{W_1 \cdot W_{2,\text{use}}}{W_1 \cdot W_{2,\text{use}} - \|W_1\|^2} \quad (3.11)$$

が得られる。これは、式 (3.9) を満たす m の閉形式解である。すなわち、 m_1 は安全勾配 W_1 との内積が 0 となる混合方向を構成するための係数として用いられる。

一方、性能勾配との直交条件

$$e \cdot W_{2,\text{use}} = 0 \quad (3.12)$$

を考えると、理想的には

$$m = \frac{\|W_{2,\text{use}}\|^2}{\|W_{2,\text{use}}\|^2 - W_1 \cdot W_{2,\text{use}}} \quad (3.13)$$

となる。しかし、実装では分子および分母に縮小前の W_2 のノルムを用いており、

$$m_2 = \frac{\|W_2\|^2}{\|W_2\|^2 - W_1 \cdot W_{2,\text{use}}} \quad (3.14)$$

としている。

したがって、 m_2 は式 (3.12) を厳密に解いたものではなく、縮小前の W_2 のノルムと縮小後の $W_{2,\text{use}}$ との内積を組み合わせた混合係数として定義される。このように、本手法では m_1 と m_2 を用いて、安全勾配と性能勾配の相対的な関係を更新方向の決定に反映させる。

3.6.3 学習進行度による混合係数の補間

学習の進行に伴って、安全性を重視する段階から性能を重視する段階へ更新方向を変化させるため、学習進行度を表す *progress* を導入する。

エピソード番号を ep 、Stage 2 のエピソード数を N_{stage2} とすると、学習進行度を

$$\text{progress}(ep) = \min \left\{ 1, \left(\frac{ep}{N_{\text{stage2}}} \right)^2 \right\} \quad (3.15)$$

と定義する。これにより、学習初期では *progress* が小さく、学習が進むにつれて 1 に近づく。

この進行度を用いて、 m_1 と m_2 の線形補間による係数 m_{prog} を

$$m_{\text{prog}} = (1 - \text{progress})m_2 + \text{progress} m_1 \quad (3.16)$$

と定義する。したがって、学習初期では m_2 の影響が大きく、学習の進行に伴って m_1 の影響が大きくなる。

3.6.4 バッチ安全度 h_{value} の算出

学習進行度だけでなく、現在の状態がどの程度安全であることを更新方向の決定に反映させるため、ミニバッチに含まれる状態の安全度から h_{value} を算出する。

まず、ミニバッチに含まれる状態を $\{s_1, s_2, \dots, s_N\}$ とし、各状態に対する安全度を

$$h_{\text{batch}} = \{h(s_1), h(s_2), \dots, h(s_N)\} \quad (3.17)$$

とする。この中から危険側の状態をより強く反映させるため、下位 15% 分位点を

$$h_{\text{quant}} = \text{Quantile}_{0.15}(h_{\text{batch}}) \quad (3.18)$$

として求める。

さらに、過去の h_{quant} を最大 80 回分保持するバッファ B を用い、その平均値を

$$h_{\text{value}} = \frac{1}{|B|} \sum_{v \in B} v \quad (3.19)$$

とする。

このように、下位 15% 分位点と移動平均を組み合わせることで、単一のミニバッチにおける一時的な変動の影響を抑えながら、危険側の状態を考慮した安全度を得ることができる。

実装では、 h_{quant} を計算した後、`h_buffer` に格納し、バッファサイズが 80 を超えた場合には最も古い値を削除する。その後、バッファ内の平均値を h_{value} として使用する。また、本実験では `stage1.episodes = 0` としているため、この処理は学習開始時から継続して実行される。

3.6.5 安全ゲートによる重み付け

学習進行度に基づく重み付けだけでは、現在の安全状態が十分に考慮されない可能性がある。そこで、 h_{value} が低下して危険側に近づいた場合に、学習進行度に依存する重み付けとは別に、安全性を重視するよう更新方向を調整する安全ゲート ω を導入する。

安全ゲートはシグモイド関数を用いて、

$$\omega = \sigma(k_{\text{gate}}(c - h_{\text{value}})) \quad (3.20)$$

と定義する。本手法では、

$$k_{\text{gate}} = 1.63, \quad c = 0.55 \quad (3.21)$$

としている。ここで、 $\sigma(\cdot)$ はシグモイド関数、 c は安全度に対する基準値、 k_{gate} はシグモイド関数の変化の大きさを調整する係数である。

式 (3.20) より、 h_{value} が基準値 c を下回るほど ω は 1 に近づき、逆に h_{value} が c を上回るほど ω は 0 に近づく。したがって、危険側に近づくほど m_1 の影響が強くなり、安全側では m_{prog} の影響が強くなる。

なお、実装ではこの ω に対応する変数として `safety_level` を用いている。

3.6.6 最終更新方向の算出

最後に、安全ゲート ω と学習進行度に基づく係数 m_{prog} を用いて、最終的な混合係数 m_{dir} を算出する。

まず、安全ゲートによる重み付けを行った係数を

$$m_{\text{dir}}^{\text{raw}} = \omega m_1 + (1 - \omega) m_{\text{prog}} \quad (3.22)$$

とする。

ただし、 $m_{\text{dir}}^{\text{raw}}$ が極端な値となることを防ぐため、 m_1 と m_2 の大小関係から定まる範囲内に制限する。すなわち、

$$m_{\text{dir}} = \text{clip}(m_{\text{dir}}^{\text{raw}}, \min(m_1, m_2), \max(m_1, m_2)) \quad (3.23)$$

とする。

そして、得られた m_{dir} を用いて、安全勾配 W_1 と縮小後の性能勾配 $W_{2,\text{use}}$ を混合し、Actor の更新に用いる方向ベクトル w_{dir} を

$$w_{\text{dir}} = m_{\text{dir}} W_1 + (1 - m_{\text{dir}}) W_{2,\text{use}} \quad (3.24)$$

として求める。

以上の処理により、安全勾配 W_1 と性能勾配 W_2 の相対的な関係に加えて、学習の進行度および現在の安全度を考慮した更新方向 w_{dir} が得られる。この更新方向は、次節で述べる勾配の大きさの調整処理への入力として用いられる。

なお、本手法は CAC で用いられる制約付き最適化問題を直接解くものではない。CAC の制約付き最適化における勾配間の幾何学的関係を参考として、 m_1 および m_2 を算出し、これらを安全度および学習進行度に応じて組み合わせることで更新方向を決定する。したがって、ここで示した一連の処理は、CAC の最適化問題の厳密解ではなく、それに着想を得て構成したヒューリスティックな勾配計算則である。

3.7 勾配の大きさの調整

3.6 節では、安全勾配 W_1 と性能勾配 $W_{2,\text{use}}$ を組み合わせることにより、Actor の更新方向 w_{dir} を決定した。しかし、更新方向が決定されただけでは、その勾配の大きさは十分に制御されていない。特に、勾配のノルムが過大な場合には、1 回の Actor 更新によるパラメータの変化が大きくなり、学習が不安定になる可能性がある。

そこで、本手法では、3.6 節で得られた w_{dir} に対して、複数段階の縮小およびクリップ処理を適用する。具体的には、未抑制係数 m_{raw} の算出、ソフト縮小、相対上限、ハードクランプ、およびノルムクリップを順番に適用する。さらに、`restricted_direction` 関数から得られた更新ベクトルに対して、Actor の更新処理側でも追加のクリップを行う。

以下では、これらの処理を実装順に説明する。

3.7.1 未抑制係数 m_{raw}

3.6 節で得られた w_{dir} は、安全勾配と性能勾配を組み合わせて決定された更新方向である。次に、この更新方向に対して適用するスケール係数を決定する必要がある。そこでまず、性能勾配 $W_{2,\text{use}}$ が w_{dir} の方向にどの程度寄与しているかを表す基準値として、未抑制係数 m_{raw} を導入する。

m_{raw} は、 $W_{2,\text{use}}$ を w_{dir} 方向へ射影したときの係数として、

$$m_{\text{raw}} = \frac{W_{2,\text{use}} \cdot w_{\text{dir}}}{\|w_{\text{dir}}\|^2} \quad (3.25)$$

と定義する。

式 (3.25) により、 $W_{2,\text{use}}$ と w_{dir} の内積および w_{dir} のノルムから、更新方向に対する性能勾配の相対的なスケールを求めることができる。この m_{raw} をそのまま更新量として使用するのではなく、以降の処理によって段階的に制限する。

実装では、

$$m_{\text{raw}} = \frac{W_{2,\text{use}} \cdot w_{\text{dir}}}{\|w_{\text{dir}}\|^2} \quad (3.26)$$

に対応する処理として、

```
m0 = torch.dot(W2_use, w_dir) / w_dir.norm()**2
```

を用いている。

3.7.2 ソフト縮小

m_{raw} が大きな値をとる場合、これをそのまま用いると更新ベクトルの大きさが過大になる可能性がある。そこで、急激な切り捨てではなく、値が大きくなるにつれて滑らかに縮小するソフト縮小処理を適用する。

ソフト縮小後の係数 m_{soft} を、

$$m_{\text{soft}} = \text{sign}(m_{\text{raw}}) \frac{|m_{\text{raw}}|}{1 + \left(\frac{|m_{\text{raw}}|}{m_{\text{thresh}}}\right)^p} \quad (3.27)$$

と定義する。本手法では、

$$m_{\text{thresh}} = 100, \quad p = 1.5 \quad (3.28)$$

としている。

ここで、 m_{thresh} は縮小の影響が強くなり始める基準値であり、 p は縮小の強さを調整する指数である。 m_{raw} の絶対値が小さい場合には、分母における追加項が小さいため、元の値に近い値が維持される。一方、 $|m_{\text{raw}}|$ が大きくなるにつれて分母が大きくなり、 m_{soft} の絶対値が抑制される。また、 $\text{sign}(m_{\text{raw}})$ を用いることで、係数の符号は維持される。

実装では、まず m_{raw} の絶対値を用いて

```
m_shrunk =  
    m0_abs / (1.0 + (m0_abs / m_thresh)**p)
```

を計算し、

```
m_soft = m0_sign * m_shrunk
```

としている。

3.7.3 相対上限

ソフト縮小を適用した後も、 m_{soft} の絶対値が過度に大きくなる可能性がある。そこで、性能勾配 $W_{2,\text{use}}$ のノルムを基準として、 m_{soft} に対する相対的な上限を設ける。

相対上限を適用した係数 m_{rel} を、

$$m_{\text{rel}} = \text{sign}(m_{\text{soft}}) \min(|m_{\text{soft}}|, k_{\text{rel}} \|W_{2,\text{use}}\|) \quad (3.29)$$

と定義する。本手法では、

$$k_{\text{rel}} = 4.5 \quad (3.30)$$

としている。

この処理により、 m_{soft} の絶対値が $W_{2,\text{use}}$ のノルムに対して過度に大きくならないように制限する。すなわち、固定値による上限だけではなく、その時点における性能勾配の大きさに応じて相対的な上限を変化させる。

実装では、

```
rel_cap = k_rel * (W2u_norm + 1e-12)  
m_rel = torch.sign(m_soft) *  
    torch.min(m_soft.abs(), rel_cap)
```

として計算している。ここで 10^{-12} は数値計算上の安定性を確保するために付加している。

3.7.4 ハードクランプ

ソフト縮小および相対上限を適用した後も、係数の絶対値が非常に大きくなる可能性がある。そこで、係数そのものに対して固定された絶対上限を設け、極端な値を最終的に制限する。

ハードクランプ後の係数 m_{clamp} を、

$$m_{\text{clamp}} = \text{clip}(m_{\text{rel}}, -M, M) \quad (3.31)$$

と定義する。本手法では、

$$M = 21.0 \quad (3.32)$$

としている。
この処理によって、

$$-M \leq m_{\text{clamp}} \leq M \quad (3.33)$$

となり、係数の絶対値に対して明示的な上限が設定される。
実装では、

```
m_scale =
    torch.clamp(m_rel, min=-M, max=M)
```

として計算している。

3.7.5 関数内最終ノルムクリップ

ここまでの処理では、主としてスケール係数 m_{clamp} の大きさを制御してきた。しかし、実際に Actor の更新に用いるのは、この係数を掛けた更新ベクトルである。そこで、最終的な更新ベクトルそのもののノルムに対しても上限を設ける。

まず、

$$\widetilde{W} = m_{\text{clamp}} w_{\text{dir}} \quad (3.34)$$

とする。そして、 $\widetilde{W}$ のノルムが上限 $W_{\text{max}}^{(1)}$ を超えた場合には、方向を維持したままノルムを上限値まで縮小する。したがって、最終的な更新ベクトル $\widehat{W}$ を

$$\widehat{W} = \begin{cases} \widetilde{W}, & \|\widetilde{W}\| \leq W_{\text{max}}^{(1)} \\ \frac{W_{\text{max}}^{(1)}}{\|\widetilde{W}\|} \widetilde{W}, & \|\widetilde{W}\| > W_{\text{max}}^{(1)} \end{cases} \quad (3.35)$$

と定義する。本手法では、

$$W_{\text{max}}^{(1)} = 400 \quad (3.36)$$

としている。

この処理は更新ベクトルの方向を変更せず、そのノルムのみを上限値以下に制限するものである。
実装では、

```
w_scaled = m_scale * w_dir

if w_scaled.norm() > max_update_norm:
    w_scaled = w_scaled * (
```

```

        max_update_norm /
        (w_scaled.norm() + eps)
    )

```

としており, `max_update_norm=400` としている.

以上の処理により, `restricted_direction` 関数内では, 3.6 節で決定した更新方向 w_{dir} に対して, スケール係数の制御と更新ベクトルのノルム制御を行い, 最終的な更新ベクトル $\widehat{W}$ を得る.

3.7.6 関数外における追加クリップ

`restricted_direction` 関数内でノルム制御を行った後, その戻り値である更新ベクトルは, Actor の更新処理に渡される. この際, Actor のパラメータへ勾配を書き込んで更新する前に, さらに 2 段階のクリップ処理を適用している. したがって, 実際に Actor のパラメータ更新に用いられる勾配は, 3.7.5 節で求めた $\widehat{W}$ に対して追加の制限を受ける.

まず, `restricted_direction` から返された更新ベクトル e に対して, ノルムの上限 $W_{\text{max}}^{(2)}$ を設ける. その処理は,

$$e' = \begin{cases} e, & \|e\| \leq W_{\text{max}}^{(2)} \\ \frac{W_{\text{max}}^{(2)}}{\|e\|} e, & \|e\| > W_{\text{max}}^{(2)} \end{cases} \quad (3.37)$$

で表される. 本実装では,

$$W_{\text{max}}^{(2)} = 102.5 \quad (3.38)$$

としている.

その後, 得られた e' を各 Actor パラメータの勾配へ展開して設定した後, PyTorch の `clip_grad_norm` を用いて, Actor 全体の勾配ノルムに対してさらに上限を設ける. この処理を

$$e'' = \text{clip_grad_norm}(e', W_{\text{max}}^{(3)}) \quad (3.39)$$

と表す. 本実装では,

$$W_{\text{max}}^{(3)} = 50.0 \quad (3.40)$$

としている.

実装上は,

```

max_e_norm = 102.5
e_norm = e.norm().item()

if e_norm > max_e_norm:
    e = e * (

```

```

        max_e_norm /
        (e_norm + 1e-12)
    )

# 各パラメータへ勾配を展開
# p.grad に代入

max_grad_norm = 50.0
torch.nn.utils.clip_grad_norm_(
    self.actor_net.parameters(),
    max_grad_norm
)

self.actor_optim.step()

```

という順序で処理される。

以上より、本手法では、更新方向を決定した後、

$$400 \rightarrow 102.5 \rightarrow 50.0 \quad (3.41)$$

という複数のノルム上限を順番に適用している。これにより、更新ベクトルの過大化を段階的に抑制する構成となっている。

ただし、これらのクリップ処理は更新量を制限するためのものであり、それ自体が CBF 条件を満たすことや、安全性を数学的に保証するものではない。また、現在保存されている実験ログには各クリップ処理の発動回数が直接記録されていないため、それぞれのクリップが実際の学習中にどの程度の頻度で適用されたかについては、現在のログから定量的に評価することはできない。

3.8 バリア関数パラメータ α のスケジューリングとエントロピー温度との関係

3.6 節では、安全度および学習の進行度に基づいて、安全勾配と性能勾配を組み合わせる方法について説明した。一方、本手法では、安全性の評価に用いる離散時間 CBF 条件に含まれるパラメータ α についても、学習の進行に応じて変化させている。

離散時間 CBF 条件は、

$$h(s_{t+1}) \geq \alpha h(s_t) \quad (3.42)$$

と表される。本研究では、この α を一定値とせず、学習エピソードに応じて変化させるスケジューリングを導入している。

なお、本実装では、この α が CBF の安全性評価だけでなく、SAC におけるエントロピー正則化

の温度パラメータとしても使用されている。したがって、本節では、まず α のスケジューリング方法を示し、その後、CBF における α と SAC における温度パラメータとの関係について説明する。

3.8.1 α のスケジューリング

学習過程における安全性と性能のバランスを変化させるため、本研究ではエピソード番号に応じて α を線形に変化させる。具体的には、エピソード ep における α を、

$$\alpha(ep) = \alpha_{\text{start}} + (\alpha_{\text{end}} - \alpha_{\text{start}}) \frac{ep}{N_{\text{total}}} \quad (3.43)$$

と定義する。

ここで、 α_{start} は学習開始時の値、 α_{end} は学習終了時の値、 N_{total} は総エピソード数を表す。本研究では、

$$\alpha_{\text{start}} = 0.1, \quad \alpha_{\text{end}} = 0.9 \quad (3.44)$$

としている。

したがって、学習の進行に伴って α は 0.1 から 0.9 へ線形に変化する。

実装では、`run_experiment` 内の各エピソード開始時に、

```
alpha_start, alpha_end = 0.1, 0.9
new_alpha = alpha_start + \
    (alpha_end - alpha_start) * (ep / total_eps)

args['alpha'] = new_alpha
agent.alpha = torch.tensor(new_alpha).to(device)
```

として α を更新している。

3.8.2 CBF における α の役割

CBF における α は、離散時間における安全条件の評価に用いられる。本研究で用いる安全報酬は、

$$r_1(s_t, a_t) = \exp(\min\{h(s_{t+1}) + (\alpha - 1)h(s_t), 0\}) - 1 \quad (3.45)$$

である。

ここで α は、現在の状態 s_t におけるバリア関数値と次状態 s_{t+1} におけるバリア関数値との関係性を評価するために用いられる。したがって、式 (3.43) によって α を変化させることで、学習の進行に応じて安全性の評価基準を変化させることになる。

本実装では、各エピソードの開始時に更新された α が、`compute_safety_reward` の引数として渡され、式 (3.45) の α として使用される。

3.8.3 SAC のエントロピー温度としての α

本実装における α は、CBF におけるパラメータだけではなく、SAC におけるエントロピー正則化の温度パラメータとしても使用されている。

SAC における Actor の目的関数は、

$$J_{\text{stab}} = \mathbb{E} [\alpha \log \pi_{\theta}(a|s) - \min(Q_1(s, a), Q_2(s, a))] \quad (3.46)$$

であり、 α は方策のエントロピーに対する重みとして作用する。

また、Navigation Critic のソフトターゲットにも、

$$y_{\text{nav}} = r_2 + \gamma \mathbf{1}_{\{\text{not done}\}} [\min(Q_1^t, Q_2^t) - \alpha \log \pi(a'|s')] \quad (3.47)$$

のように α が用いられている。

したがって、本実装では、エピソード開始時に設定された `args['alpha']` および `agent.alpha` が、CBF の安全性評価だけでなく、SAC におけるエントロピー正則化にも共通して使用されている。

3.8.4 自動温度調整との関係

SAC では、エントロピー温度を固定値とする代わりに、目標エントロピーに基づいて自動的に調整することができる。本実装においても、`SoftActorCriticModel` は `log_alpha` および `alpha_optim` を保持しており、Actor および Critic の更新処理の中で温度パラメータを自動調整している。

具体的には、温度パラメータに対する損失として、

$$L_{\alpha} = -\mathbb{E} [\log \alpha (\log \pi_{\theta}(a|s) + \mathcal{H}_{\text{target}})] \quad (3.48)$$

を用いている。

実装では、

```
alpha_loss = -(
    self.log_alpha *
    (logp_pi + self.target_entropy).detach()
).mean()
```

```
self.alpha_optim.zero_grad()
alpha_loss.backward()
self.alpha_optim.step()
```

```
self.alpha = self.log_alpha.exp()
```

としており、Actor の更新ごとに自動温度調整を行っている。

一方、`run_experiment` では、各エピソードの開始時に式 (3.43) によって計算した α を

```
args['alpha'] = new_alpha
agent.alpha = torch.tensor(new_alpha).to(device)
```

として再設定している．そのため，SAC の自動温度調整によって更新された `agent.alpha` は，次のエピソード開始時にスケジューリングによって計算された値で書き換えられる．

以上より，本実装では，エピソード内では SAC の自動温度調整が作用する一方で，エピソードの境界では式 (3.43) による手動設定が行われる構造となっている．

このように，本実装では同一の α が，

1. CBF における安全性評価のパラメータ，
2. SAC におけるエントロピー温度，

という異なる役割に共通して使用されている．また，SAC の自動温度調整とエピソード単位の手動スケジューリングが併存している．

ただし，この 2 つの調整機構が学習結果に対して具体的にどのような影響を与えるかについては，現在の実験結果のみからは明確に判断できない．そのため，本研究ではこの構造を実装上の特徴として記述し，両者の相互作用による因果的な影響については別途検証すべき課題とする．

3.9 提案手法の学習アルゴリズム

3.6 節から 3.8 節では，安全勾配と性能勾配に基づく更新方向の決定，更新量の調整，およびバリア関数パラメータ α のスケジューリングについて説明した．本節では，これらの処理を組み合わせた提案手法全体の学習アルゴリズムを示す．

提案手法では，SAC を基礎とし，Safety Critic と Navigation Critic をそれぞれ Twin-Q 構造で学習する．各エピソードの開始時に α を更新し，環境との相互作用によって得られた遷移を Replay Buffer へ蓄積する．Replay Buffer に十分なデータが蓄積された後は，Critic の更新と Actor の更新を繰り返す．

Actor の更新では，安全損失と性能損失からそれぞれ安全勾配 W_1 と性能勾配 W_2 を計算する．その後，3.6 節で述べた `restricted_direction` によって両勾配を組み合わせた更新方向を決定し，3.7 節で述べた複数段階のノルム制御を適用した上で Actor のパラメータを更新する．

以下に，`Cleaned.ipynb` の `run_experiment` および `update_critics_and_actor` の処理順序に対応した提案手法の学習アルゴリズムを示す．

Algorithm 1 提案手法の学習アルゴリズム

Require: Actor π_θ , Navigation Critic Q_{nav} , Safety Critic Q_{safe} , Replay Buffer $\mathcal{D}$, experiment parameters

```
1:  $run\_seed \leftarrow BASE\_SEED + run\_id$ 
2: Initialize environment, Actor, Critics, and Replay Buffer
3: for  $ep = 1, \dots, N_{\text{total}}$  do
4:    $\alpha(ep) \leftarrow \alpha_{\text{start}} + (\alpha_{\text{end}} - \alpha_{\text{start}}) \frac{ep}{N_{\text{total}}}$ 
5:   Set args['alpha']  $\leftarrow \alpha(ep)$  and agent.alpha  $\leftarrow \alpha(ep)$ 
6:   Reset environment with  $run\_seed$  for the first episode
7:   while episode is not terminated do
8:     Select action  $a$  by random exploration if  $n_{\text{steps}} < start\_steps$ ; otherwise sample  $a \sim \pi_\theta(\cdot|s)$ 
9:     Record  $h(s)$  for logging
10:    if  $|\mathcal{D}| > batch\_size$  then
11:      Sample a mini-batch from  $\mathcal{D}$ 
12:      Compute Navigation Critic target  $y_{\text{nav}}$ 
13:      Compute Safety Critic target  $y_{\text{safe}}$ 
14:      Update  $Q_{\text{nav}}$  and  $Q_{\text{safe}}$ 
15:      Compute performance gradient  $W_2 = \nabla_\theta J_{\text{stab}}$ 
16:      Compute safety gradient  $W_1 = \nabla_\theta J_{\text{safe}}$ 
17:      if  $ep \leq stage1\_episodes$  then
18:        Update Actor using the safety loss only
19:      else
20:        Compute  $h_{\text{value}}$  from the lower 15% quantile of the batch safety values
21:        Compute learning progress  $progress = \min \left( 1, \left( \frac{ep}{stage2\_episodes} \right)^2 \right)$ 
22:         $(e, m_{\text{dir}}) \leftarrow \text{restricted\_direction}(W_1, W_2, progress, h_{\text{value}})$ 
23:        Apply soft shrink, relative cap, and hard clamp to the scale coefficient
24:        Apply the first norm clipping with  $W_{\text{max}}^{(1)} = 400$ 
25:        Apply the second norm clipping with  $W_{\text{max}}^{(2)} = 102.5$ 
26:        Expand  $e$  to the Actor parameter gradients
27:        Apply the final gradient norm clipping with  $W_{\text{max}}^{(3)} = 50$ 
28:        Update Actor parameters
29:      end if
30:      Update entropy temperature  $\alpha$  automatically
31:      Soft-update target Critics
32:    end if
33:    Execute action  $a$  in the environment and obtain  $s'$  and  $r_{\text{nav}}$ 
34:    Compute safety reward  $r_{\text{safe}}$  using  $\alpha(ep)$ 
35:    Count safety violation if  $h(s') < 0$ 
36:    Store  $(s, a, r_{\text{nav}}, r_{\text{safe}}, s', done)$  in  $\mathcal{D}$ 
37:     $s \leftarrow s'$ 
38:  end while
39:  Record episode statistics
40: end for
```

Replay Buffer へのデータ蓄積

各エピソードでは、現在の状態 s から Actor によって行動 a を選択し、環境へ入力する。環境から次状態 s' および Navigation reward r_{nav} を取得した後、3.8 節で説明した $\alpha(ep)$ を用いて Safety reward r_{safe} を計算する。

さらに、次状態におけるバリア関数値が

$$h(s') < 0 \quad (3.49)$$

となった場合には、安全性違反としてカウントする。

得られた遷移

$$(s, a, r_{\text{nav}}, r_{\text{safe}}, s', \text{done}) \quad (3.50)$$

を Replay Buffer $\mathcal{D}$ へ保存する。この処理をエピソード終了まで繰り返す。

Critic の更新

Replay Buffer に蓄積されたデータ数が `batch_size` を超えると、ミニバッチをサンプリングして Navigation Critic および Safety Critic を更新する。

Navigation Critic では、式 (3.47) で示した SAC のソフトターゲット y_{nav} を用いて、Twin-Q 構造を持つ 2 つの Q ネットワークを更新する。同様に、Safety Critic では式 (??) で示した y_{safe} を用いて 2 つの Q ネットワークを更新する。

本実験条件では、

$$\text{stage1_episodes} = 0 \quad (3.51)$$

としているため、Critic 更新において Stage 1 と Stage 2 を切り替える実質的な分岐は存在せず、Navigation Critic と Safety Critic の両方が継続的に更新される。

Actor の更新

Critic の更新後、Actor の更新を行う。まず、Actor の性能損失 J_{stab} と安全損失 J_{safe} についてそれぞれ勾配を計算し、

$$W_1 = \nabla_{\theta} J_{\text{safe}}, \quad W_2 = \nabla_{\theta} J_{\text{stab}} \quad (3.52)$$

を得る。

本実験では $\text{stage1_episodes} = 0$ としているため、実際の実験では Stage 1 の安全損失のみを用いる分岐は実行されず、エピソード 1 から W_1 と W_2 の両方を用いた更新方向の計算が行われる。

まず、バッチ内の安全度および学習進行度を用いて、3.6 節で説明した `restricted_direction` により更新方向を決定する。この処理では、 W_2 の縮小、 m_1 および m_2 の計算、進行度による補間、安全ゲートによる調整を経て、最終的な方向ベクトル w_{dir} を構成する。

その後、3.7 節で説明したスケール係数の縮小および複数段階のノルム制御を適用する。得られた更新ベクトルを Actor の各パラメータの勾配へ展開し、最終的な勾配クリップを適用した後、Optimizer の更新処理によって Actor のパラメータを更新する。

学習初期におけるデータ収集

学習開始直後は Replay Buffer に十分なデータが蓄積されていないため、条件

$$|\mathcal{D}| > batch_size \quad (3.53)$$

を満たさない。本実装ではこの条件が満たされるまで Critic および Actor の更新を行わず、環境との相互作用によるデータ収集のみを行う。

また、初期の一定ステップでは、Actor による行動選択ではなく環境の `action_space.sample()` によるランダム行動を用いることで、Replay Buffer へ多様な経験を蓄積する。

Stage 1 分岐の扱い

本実装には、CAC の Stage 1 に対応する安全損失のみを用いた Actor 更新処理がコード上に残されている。しかし、本研究の実験条件では、

$$stage1_episodes = 0 \quad (3.54)$$

としている。また、エピソード番号は 1 から開始するため、

$$ep \leq stage1_episodes \quad (3.55)$$

の条件はすべてのエピソードにおいて成立しない。

したがって、本研究の実験では Stage 1 分岐は実行されず、エピソード 1 から安全勾配 W_1 と性能勾配 W_2 を用いた提案手法による Actor 更新が行われる。

このように、Stage 1 に対応するコードを残しつつ、実験条件として Stage 1 の実行期間を 0 とすることで、CAC における二段階学習を用いず、学習開始時から安全性と性能を同時に考慮する構成としている。

以上の処理を各エピソードについて繰り返すことで、Safety Critic および Navigation Critic を更新しながら、安全勾配と性能勾配を組み合わせた Actor 更新を継続的に行う。

本実装では、Stage 1 を実行しない実験条件を採用しているため、従来の CAC のように安全学習から性能学習へ明示的に切り替えるのではなく、学習開始時から安全性と性能の両方を考慮した Actor 更新を行う。さらに、更新方向の決定には安全度と学習進行度を利用し、その後の更新量についても複数段階のノルム制御を適用する。

これにより、本研究では、CAC で用いられる制約付き最適化を直接実行する代わりに、勾配の内積、ノルム、安全度および学習進行度に基づいて更新方向と更新量を逐次的に決定する構成としている。

第 4 章

提案手法の検証

本章では，第 3 章で提案した安全度・進行度に基づく勾配混合手法およびノルム制御を用いた方策更新について，Pendulum-v1 環境を用いて学習性能を評価する．まず，実験に用いた環境設定，ネットワーク構成，報酬設計，安全制約，ノルム制御の各要素について述べる．続いて，学習曲線および安全性指標の推移を示し，提案手法の有効性を検証する．

4.1 実験設定

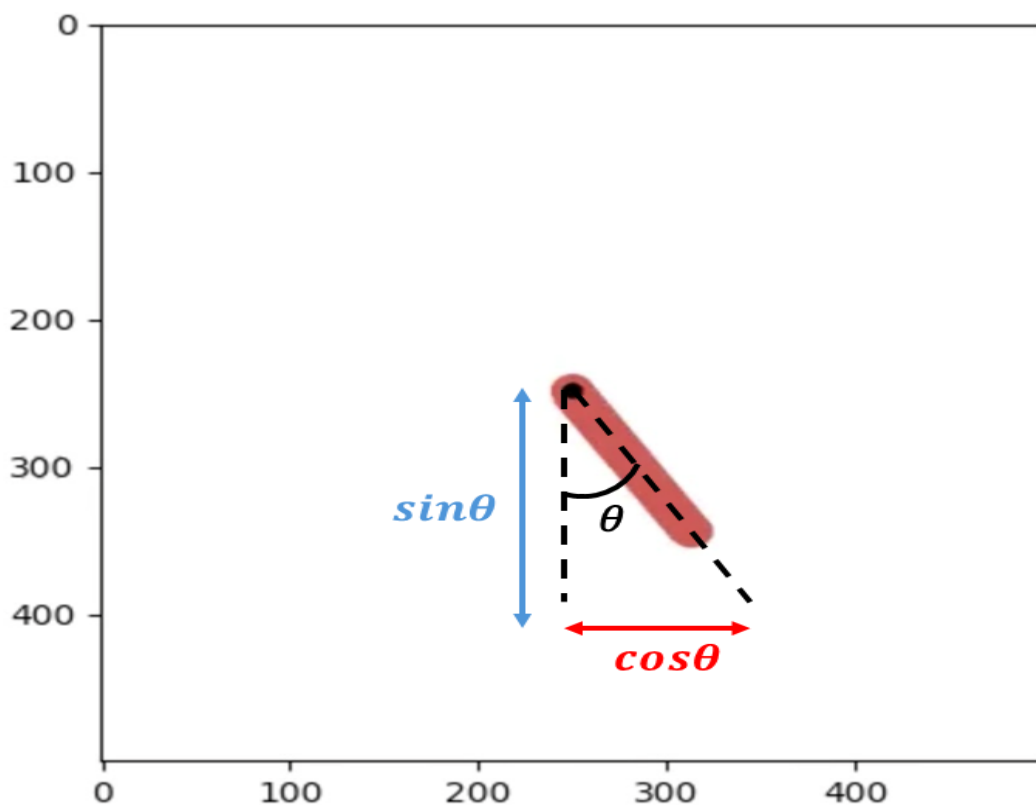


図 4.1 Pendulum-v1

Pendulum-v1 環境での振り子の状態は，図 4.1 に示すように，支点を中心とした振り子の角度 θ に基づいて定義される．ただし，本環境では角度 θ をそのまま状態として用いるのではなく， $[\sin \theta, \cos \theta, \dot{\theta}]$ の 3 次元ベクトルとして状態が与えられる．角度 θ は振り子が鉛直下向きに静止している状態を基準として，支点を中心に反時計回りを正として定義される．Actor と Critic はいずれも三層の深層ニューラルネットワークから構成し，中間層のニューロン数は 256 個である．制御バリア関数は

$$h(\theta) = \dot{\theta} + 2.0$$

と定義した．

4.2 学習結果

4.3 考察

第 5 章

おわりに

5.1 まとめ

本研究では、システムの振る舞いが状態空間上で指定された安全領域の中に留まり続けるという安全仕様を満たしつつシステムを安定化する学習手法として、certified actor critic (CAC) を扱った。CAC ではまず安全仕様を満たす方策を強化学習で獲得し、その後、安全性と安定化を別個の critic ネットワークで評価して得られる勾配を基に actor を更新する枠組みを採用した。本研究では、その勾配計算を簡易化する手法を 3 つ提案し、さらに学習途中で確認された安全性の低下を改善するために勾配の更新方向を変更した改良手法を導入し、シミュレーションにより学習能力を評価した。

第 1 章では研究背景と課題を整理し、安全性を考慮した強化学習の必要性を述べた。第 2 章では SAC と CAC の理論的基盤を概説し、制御バリア関数や制御 Lyapunov 関数に基づく安全性・安定性の評価方法を示した。第??章では、CAC における actor 更新で用いる勾配の簡易計算法として方法 1~3 を提案し、各手法の定式化と数値的な工夫を詳述した。第??章では、提案手法のうち勾配の寄与を学習進行に応じて段階的に性能寄りへシフトさせる改良手法を提示し、第??章と同一の Pendulum-v1 環境でシミュレーションを行ってその有効性を検証した。シミュレーション結果から、本改良手法は方法 1~3 と比較して、安全性を比較的維持しつつ到達性能の改善を達成できることを確認した。

5.2 今後の課題

今後の課題としては、まず複数の安全制約を同時に扱う拡張が挙げられる。現行手法は単一のバリア関数に基づく安全性評価を前提としている。したがって、複数の相互に関連する安全条件を満たしつつ性能を向上させるための勾配合成則やスケジューリングの設計が必要である。また、移動ロボットなど実機への応用も重要な課題である。実機環境では観測ノイズやモデル誤差、計算資源の制約が存在するため、提案手法の計算効率向上や実装上の工夫が求められる。これらの課題に取り組むことで、本研究の枠組みをより実用的で汎用的な安全制御学習手法へと発展させることが期待される。

謝辞

本研究を進めるにあたり，多大なるご指導をいただきました潮俊光教授に深く感謝申し上げます．先生方の的確なご指摘と温かいご支援がなければ，本研究をここまで完成させることはできませんでした．また，実験や議論にご協力いただいた研究室の皆様にも深く感謝します．最後に，本研究期間日常生活ならびに精神面において支援を受けた家族に心より感謝の意を表します．

参考文献

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, 2018.
- [2] 牧野, 澁谷, 白川編著, これからの強化学習, 4 章, 森北出版, 2016.
- [3] M. Sewak, Deep Reinforcement Learning, Frontiers of Artificial Intelligence, Springer, 2019.
- [4] 小林泰介, 詳解強化学習の発展と応用, 科学情報出版株式会社, 2024.
- [5] 長孝之, 強化学習アルゴリズム, コロナ社, 2025.
- [6] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” Nature vol. 518, pp. 529-533, 2015.
- [7] R. S. Sutton, *et al.*, “Policy gradient methods for reinforcement learning with function approximation,” Proc. NIPS1999, pp. 1057-1063, 1999.
- [8] J. Schulman, *et al.*, “Trust Region policy optimization,” Proc. 32nd ICML, pp. 1889-1897, 2015.
- [9] J. Schulman, *et al.*, “Proximal policy optimization,” arXiv preprint arXiv:1707.06347, 2017.
- [10] T. Haarnoja, *et al.*, “Soft actor-critic algorithms and applications,” arXiv preprint arXiv:1812.05905, 2018.
- [11] A. Wachi and Y. Sui, “Safe reinforcement learning in constrained Markov decision processes,” Proc. 37 ICML, pp. 9797-9806, 2020.
- [12] Y. Liu, A. Halev, and X. Liu, “Policy learning with constraints in model-free reinforcement learning: survey,” Proc. IJCAL-21, pp. 4508-4515, 2021.
- [13] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” IEEE Trans. Automatic Control, vol. 62, no. 8, pp. 3861-3876, 2017.
- [14] A. D. Ames, A. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: theory and applications,” Proc. 2019 18th European Control Conference, pp. 3420-3431, 2019.
- [15] A. Agrawal and K. Sreenath, “Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation,” Proc. 2017 Robotics: Science and Systems, 2017.
- [16] M. Mizoguchi and T. Ushio, “Abstraction-based symbolic control barrier functions for safety-critical embedded systems,” IEEE Control Systems Letters, vol. 6, pp. 1436-1441,

2022.

- [17] D. S. Kushwaha and Z. A. Biron, “A review on safe reinforcement learning using Lyapunov and barrier frunctions,” arXiv preprint arXiv:2508.09128, 2025.
- [18] J. Xie, S. Zhao, L. Hu, and H. Gao, “Certificated Actor-Critic: Hierarchical Reinforcement Learning with Control Barrier Functions for Safe Navigation,” Proc. IEEE ICRA2025, pp. 5482-5488, 2025.
- [19] Y. Yao, Z. Liu, Z. Cen, P. Huang, T. Zhang, W. Yu, D. Zhao, “Gradient shaping for multi-constraint safe reinforcement learning,” arXiv preprint arXiv:2312.15127, 2023.
- [20] X. Yuan, Z. Shang, W. Huang, Y. Cui, D. Chen, M. Zhu, “Effective reinforcement learning control using conservative soft actor-critic,” arXiv preprint arXiv:2505.03356, 2025.

研究業績

1. 佐藤達志，潮俊光：Certified Actor Critic 法における勾配の簡易計算法，電子情報通信学会技術報告書，vol. 125, no. 310, pp. 55-59, 2026.